

metaDEBASS: Per-alert classifier outputs

A coordination ask for broker collaborators

Xianzhe (TZ) Tang
tztang@bu.edu
Boston University

April 27, 2026

Abstract

metaDEBASS is an early-epoch meta-classifier for Rubin/LSST. It combines broker classifier outputs with locally re-run light-curve experts so we can rank transients for spectroscopic follow-up after only a few detections. The strict rule is simple: a row for N detections may use only information available at those N detections, not anything learned later.

For several brokers we can query only the current object-level score. That is useful for live triage, but not for training early-epoch models: the current score may have been computed after many more detections than the training row is allowed to see. This memo lists which classifier outputs are safe now, which are blocked by timing, and what we would ask broker teams to expose.

1 Why per-alert outputs matter

For one transient at one alert epoch, we want to predict a follow-up priority: likely Type Ia, interesting non-Ia, or neither. The training table is keyed by (`diaObjectId`, `n_det`): one row per object per detection count from the first detection to the latest.

Every column in that row has to be *causally safe*. It must be derivable from the alert stream and broker queries truncated at N detections. Light-curve features satisfy this by construction because we re-extract them from truncated photometry. External classifier scores satisfy it only when the broker exposes one of:

1. a *per-alert* endpoint that returns the score assigned to alert k when alert k was published, for $k = 1, \dots, N$; or
2. a *timestamped* score history that lets us keep only scores whose timestamp is on or before detection N .

If the only available value is “the broker’s latest score for this object today”, the score may reflect detections beyond N . Using it at $N = 3$ when the broker had already seen $N = 15$ is decision-time leakage. The label is not leaking directly, but the model sees a future view of the object, and that biases every metric we report.

Consequence for early-epoch prioritisation. Without per-alert outputs, we either drop the broker score from training or re-run a local copy of the same method at the proper epoch cutoffs. Local re-runs work for public models such as ALeRCE LC or SuperNNova, but they duplicate broker infrastructure and do not reproduce a broker’s production preprocessing. They also cannot stand in for proprietary or unreleased classifiers.

2 Temporal tags used internally

Every event written to the broker-event table carries a `temporal_exactness` tag with one of four values:

- **exact_alert**: the score was issued for a specific alert, so we can place it at the correct epoch.
- **rerun_exact**: we re-ran the classifier locally with the photometry truncated to exactly N detections.
- **static_safe**: the output does not depend on later light-curve history, for example an image stamp classifier or a host-context flag.
- **latest_object_unsafe**: only a current object snapshot is available, with no per-alert or timestamped history. The no-leakage gold builder filters these rows by default (see `src/debass\_meta/ingest/gold.py`, around `select_events_asof()`).

The last bucket is the one we want to shrink.

3 Reality check before the table

I checked the names in the table against the local registry (`src/debass\_meta/projectors/base.py`), local metrics, and public documentation or papers. The classifier families are real; the important distinction is whether *we* currently use them in a no-leakage training run.

External checks used for the table:

- Fink ZTF fields `rf_snia_vs_nonia`, `snn_snia_vs_nonia`, and `sln_score` are documented Fink science-module outputs: https://doc.ztf.fink-broker.org/en/latest/broker/science_modules/. Fink’s LSST-facing CATS and early-SN classifiers are described in Fraga et al. 2024: <https://arxiv.org/abs/2404.08798>.
- ALeRCE exposes object probabilities and classifier metadata via its Python client, and its public papers describe the stamp, light curve, BHRF, and ATAT classifiers: <https://alerce.readthedocs.io/en/latest/apis.html> and <https://science.alerce.online/publications/>.
- Lasair Sherlock is a documented sky-context classifier: https://lasair.readthedocs.io/en/main/core_functions/sherlock.html.
- Pitt-Google documents its SuperNNova classifier and BigQuery output path: <https://pitt-broker.readthedocs.io/en/latest/broker/broker-overview.html>.
- Babamul/BOOM, ANTARES Superphot+, AMPEL SNGuess/FollowMe/FinalBet, ParSNIP, SuperNNova, ORACLE, SALT3, UPSILOn, and the `light-curve` package were checked against their papers or package documentation. For AMPEL specifically, Nordin et al. 2025 defines three LSST workflows: SNGuess for infant transient filtering, FollowMe for follow-up sample selection, and FinalBet for final classification with priors: <https://arxiv.org/abs/2501.16511>. The original ZTF SNGuess paper is Miranda et al. 2022, A&A 665, A99: <https://www.aanda.org/articles/aa/abs/2022/09/aa43668-22/aa43668-22.html>.

The AUC values below come from the latest per-expert metrics artifact in this checkout, `reports/metrics/expert\_trust\_metrics\_safe\_v6e2.json`, unless a row says “planned” or “registered”. That file has 11 measured per-expert reliability models. A v7 job script exists and is set up to add `fink/slsn` and `ampel/snguess`, but this checkout does not contain `reports/metrics/expert\_trust\_metrics\_safe\_v7.json`. The local SNGuess model files are present under `artifacts/local\_experts/snguess/`, but `xgboost` is not installed in the local Python environments I checked, so the vendored SNGuess wrapper does not load here.

4 Current state, classifier by classifier

Twenty-eight classifier or context keys are registered in `src/debass\_meta/projectors/base.py`. The table lists each key, its timing status, and what it is for. “Used a lot” here means one of two things: either the broker produced many rows in the local `data/silver/broker\_events.parquet`, or the latest safe metrics file contains a measured per-expert reliability model with a nontrivial test set.

- **Survey:** “ZTF”, “LSST”, or “any”.
- **Temporal:** one of `exact_alert`, `rerun_exact`, `static_safe`, or `latest_unsafe`.
- **Per-alert?:** “Yes” if the broker exposes alert-resolved scores; “Local” if the score comes from our re-run on truncated photometry; “N/A” if the output is static context; “No” if only a latest object snapshot is available.
- **Current use:** measured reliability model, blocked by timing, feature-only, registered, or planned.

Broker	Classifier key	Survey	Temporal	Per-alert?	Current use	Notes
Fink	<code>fink/snn</code>	ZTF	<code>exact_alert</code>	Yes	Reliability model (AUC 0.92)	Real Fink ZTF field <code>snn_snia_vs_nonia</code> . SuperNNova Ia-vs-non-Ia score, one of our main ZTF signals: 509,532 silver rows across 8,602 objects; 23,556 safe test rows in v6e.2.
Fink	<code>fink/rf_ia</code>	ZTF	<code>exact_alert</code>	Yes	Reliability model (AUC 0.81)	Real Fink ZTF field <code>rf_snia_vs_nonia</code> . Random-forest early/rising SN Ia score. Used often, but weaker than SNN: 254,766 silver rows; 23,556 safe test rows.
Fink	<code>fink/slsn</code>	ZTF	<code>exact_alert</code>	Yes	Planned / registered	Real Fink ZTF field <code>slsn_score</code> . Meant as an SLSN/SN-filter signal, not an Ia classifier. Newly registered here; absent from the local silver table and no local v7 metrics file is present. Fink returns <code>-1</code> when there are not enough points or the object is too young.
Fink LSST	<code>fink_lsst/snn</code>	LSST	<code>exact_alert</code>	Yes	Reliability model (AUC 0.84)	Real LSST-era Fink SuperNNova-style score. Heavily backfilled in this checkout: 857,832 silver rows across 3,997 objects; 9,800 safe test rows. Useful, but the LSST label mix is still weak-label heavy.
Fink LSST	<code>fink_lsst/cats</code>	LSST	<code>exact_alert</code>	Yes	Reliability model (AUC 0.85)	CATS is Fink's broad LSST classifier. Used heavily in silver (853,760 rows). In our ternary projection it mostly works as a broad transient/contaminant signal rather than a clean Ia probability.
Fink LSST	<code>fink_lsst/early_snia</code>	LSST	<code>exact_alert</code>	Yes	Reliability model (AUC 0.86)	Real Fink early-SN Ia score. Sparse by design: 12,895 silver rows and only 142 safe test rows, because it often does not trigger in the first few detections.
ALeRCE	<code>alerce/lc_classifier_transient</code>	ZTF	<code>latest_unsafe</code>	No	Blocked by timing	Real ALeRCE legacy light-curve classifier. We have 15,536 local silver rows, but they are current object snapshots, so the no-leakage build drops them. Use the local <code>alerce_lc</code> re-run instead.
ALeRCE	<code>alerce/lc_classifier_BHRF_forced_phot_transient</code>	ZTF	<code>latest_unsafe</code>	No	Blocked by timing	Real ALeRCE BHRF forced-photometry transient classifier. We have 12,946 silver rows across 2,132 objects, but no per-alert history, so it is not used for safe training.

Broker	Classifier key	Survey	Temporal	Per-alert?	Current use	Notes
ALeRCE	alerce/lc_classifier_BHRF_forced_phot_top	ZTF	latest_unsafe	No	Blocked by timing	Real ALeRCE BHRF top-level class grouping. Current snapshot only; 6,446 silver rows. Useful for live inspection, not safe as an early-epoch training feature.
ALeRCE	alerce/LC_classifier_ATAT_forced_phot(beta)	ZTF	latest_unsafe	No	Blocked by timing	Real ALeRCE ATAT transformer classifier. It is common in the raw pull (119,396 silver rows), but the public query gives latest object probabilities, so we exclude it from no-leakage training by default.
ALeRCE	alerce/stamp_classifier	ZTF	static_safe	N/A	Registered / use carefully	Real ALeRCE image-stamp CNN. It is safe only when treated as first-alert image context. This local silver has mixed legacy tags: 9,475 static-safe rows plus 38,835 older latest-snapshot rows, so normalization matters before using it.
ALeRCE	alerce/stamp_classifier_2025_beta	ZTF	static_safe	N/A	Registered / use carefully	Real newer ALeRCE stamp classifier. Same caution as the legacy stamp row: 3,384 static-safe rows plus 15,330 latest-snapshot rows in the local silver table.
ALeRCE	alerce/stamp_classifier_rubin_beta	LSST	static_safe	N/A	Registered	Real Rubin/LSST stamp beta key from ALeRCE. Only 5 local silver rows for one object here, so it is not yet a measured contributor.
Lasair	lasair/sherlock	any	static_safe	N/A	Context feature	Real Lasair Sherlock sky-context output. It is not a posterior over transient classes; it gives host/star/AGN-like context. Local silver has 50 rows, so it is a useful feature when present but not a standalone classifier in our current run.
Babamul	babamul	any	static_safe	N/A	Context feature / registered	Real Babamul/BOOM broker path. We ingest six flags such as star, rock, stationary, and cross-survey match. This is contaminant context, not a probability classifier; no local measured reliability model yet.
Pitt-Google	pittgoogle/supernnova_lsst	LSST	exact_alert	Yes	Reliability model (AUC 0.91)	Real Pitt-Google SuperNNova BigQuery output with <code>kafkaPublishTimestamp</code> . Used a lot in silver (866,838 rows across 3,998 objects); 6,552 safe test rows in v6e.2.

Broker	Classifier key	Survey	Temporal	Per-alert?	Current use	Notes
Pitt-Google	pittgoogle/supernnova_ztf	ZTF	latest_unsafe	No	Blocked by timing	Real Pitt-Google ZTF SuperNNova table. Local silver has 22,028 rows, but the table path used here lacks an alert-time column, so we cannot align scores to N detections safely.
Pitt-Google	pittgoogle/upsilon_lsst	LSST	static_safe	N/A	Registered	UPSILoN is a real periodic-variable classifier. It is useful for rejecting variable-star contaminants, not for SN subtype probabilities. The Pitt-Google LSST table path is registered here, but this checkout has 0 local rows.
ANTARES	antares/oracle	any	exact_alert	In principle	Dormant	ORACLE is a real 19-class LSST photometric classifier. ANTARES integration is registered, but no local rows or metrics are present here. Treat this as an integration target, not a current result.
ANTARES	antares/superphot_plus	any	exact_alert	In principle	Reliability model in v6e.2 (AUC 0.66)	Superphot+ is real and runs via ANTARES. The safe v6e.2 metrics file has 180 test rows, but the current local broker_events.parquet has no ANTARES rows; availability depends on the ANTARES backfill.
AMPEL	ampel/snguess	any	rerun_exact	Local or API	Registered; not active here	SNGuess is real: Miranda et al. 2022 built it as a boosted-tree filter for young extragalactic transients, and AMPEL publishes ZTF candidates to TNS as AMPEL_ZTF_NEW. In this repo the same key can mean a local vendored T2BrightSNProb re-run or AMPEL T2 output. Locally verified status: 0 rows in silver/gold, and the wrapper does not load without xgboost . Do not cite the slide-deck AUC 0.95 until the v7 metrics file is present.

Broker	Classifier key	Survey	Temporal	Per-alert?	Current use	Notes
AMPEL	ampel/parsnip_followme	any	exact_alert	Token API	Dormant	FollowMe is real, but it is not FinalBet. In Nordin et al. 2025, FollowMe uses the SNGuess-selected stream plus ParSNIP/LightGBM to choose a follow-up sample, mainly for uncertain SN Ia vs SN Ibc cases. The host/redshift/rate priors belong to FinalBet, which is not registered in this repo. No local AMPEL rows or safe metrics are present.
Local	parsnip	any	rerun_exact	Local	Dormant	ParSNIP is a real published transient light-curve model. Our wrapper needs compatible <code>model.pt</code> and <code>classifier.pkl</code> ; no measured local contribution in this checkout.
Local	supernnova	any	rerun_exact	Local	Reliability model (AUC 0.91)	Local SuperNNova re-run on truncated light curves. Used in the safe v6e.2 run with 34,793 test rows; also available on every row of the local DP1 50k snapshot file.
Local	alerce_lc	any	rerun_exact	Local	Reliability model (AUC 0.88)	Local re-run of the ALERCE light-curve classifier at proper epoch cutoffs. This is the safe substitute for the blocked ALERCE object-snapshot probabilities; 30,887 safe test rows in v6e.2.
Local	salt3_chi2	any	rerun_exact	Local	Reliability model (AUC 0.90)	SALT3 χ^2 Type-Ia consistency score using <code>sncosmo</code> . Physics-motivated and useful when the light curve has enough points; 37,675 safe test rows in v6e.2.
Local	lc_features_bv	any	rerun_exact	Local	Reliability model (AUC 0.88)	Bazin/Villar parametric-feature expert using the <code>light-curve</code> package. Sparse on short DP1 light curves but measured on 39,998 safe test rows in v6e.2.
Local	oracle_lsst	LSST	rerun_exact	Local	Dormant	Local ORACLE wrapper. The model is real, but our DP1 smoke test showed domain mismatch, with nearly all probability mass going to “other”. Not used as a current result.

Roll-up by current use. Measured in the latest safe metrics file: 11 keys (`fink/snn`, `fink/rf_ia`, `fink_lsst/snn`, `fink_lsst/cats`, `fink_lsst/early_snia`, `pittgoogle/supernnova_lsst`, `antares/superphot_plus`, `supernnova`, `alerce_lc`, `salt3_chi2`, `lc_features_bv`). Blocked by timing: 6 rows, mainly ALeRCE object snapshots plus Pitt-Google ZTF SuperNNova. Registered or dormant: the rest. This is different from saying the classifiers are not real: most are real external classifiers, but some are not usable in our current no-leakage training build.

5 Asks for the upcoming meeting

The structural ask is the same for every broker team: *do you expose, or could you expose, the score assigned to alert k at the time alert k was published?* Concretely:

ALeRCE. Is there an internal per-alert table behind the light-curve classifiers that the public REST API does not currently expose? If not, would you be willing to publish a timestamped score history per object so downstream users can filter to “score as of detection N ”? This is the largest gap for us. The local silver table has hundreds of thousands of ALeRCE object-probability rows, including ATAT and BHRF forced-photometry outputs, but the no-leakage build has to drop them.

Pitt-Google ZTF table. The LSST BigQuery table already carries `kafkaPublishTimestamp` per alert. Could the equivalent timestamp be added to the public ZTF SuperNNova table? Without it, we cannot reconstruct the per-alert score history even though the underlying model is a real and useful one.

ANTARES and AMPEL. We would like guidance on the preferred way to pull per-alert tags at scale. The classifiers are real; our current blocker is reliable bulk access and integration, not whether the methods exist. For AMPEL, the live Swagger page is reachable at <https://ampel.zeuthen.desy.de/api/live/docs>, and the OpenAPI schema exposes token-protected routes such as `/stock/ztf/{ztf_id}` and `/stock/{stock_id}/t2`. The local adapter still points at `/api/archive/object/.../t2`, which returned 404 in my check. Before we claim AMPEL coverage, we should confirm the token, the correct route, and whether the returned T2 documents include enough state time or detection-count information to align scores to N detections.

Brokers running classifiers we already re-run locally. A local substitute is useful, but it is not the broker’s deployed score. It misses deployment-specific calibration, preprocessing, and versioning. If a broker can publish per-alert scores, we would rather use the broker’s production output. ALeRCE LC is the immediate example.

Brokers running proprietary or unreleased classifiers. These are the cases where local substitution is impossible. Per-alert access matters most here.

6 What we offer back

In return, we can share, before any public release:

- Reliability curves for each contributing classifier on a held-out matched cohort, computed under the no-leakage rule.

- Per-classifier ranking and calibration metrics (ROC-AUC, expected calibration error, Brier score) at $N = 3, 5, 7, 10$ detections.
- The labelled cohort itself, with truth separated by source quality: TNS spectroscopic, TNS-name-only, broker consensus, and host-context labels.
- The schema and naming contract our pipeline already uses, so a new per-alert endpoint can plug in without a redesign on either side.

7 How to verify the numbers in this memo

The registry can be checked locally in `src/debass\_meta/projectors/base.py`. The tables used for the row counts and timing checks are:

- `data/silver/broker\_events.parquet`
One row per broker-object-score, with `temporal_exactness`.
- `data/gold/object\_epoch\_snapshots\*.parquet`
The no-leakage epoch tables. Per-classifier availability lives in the `avail__<sanitized_key>` columns.
- `reports/metrics/expert\_trust\_metrics\_safe\_v6e2.json`
The latest per-expert safe metrics artifact present in this checkout.

The gating logic lives in `src/debass\_meta/ingest/gold.py` in `select_events_asof()`. The flag `-allow-unsafe-latest-snapshot` is for scoring-time experiments only; it should not be used for training the per-expert reliability models.

8 Next steps

After the meeting we should write down three things: which broker teams to approach first, the exact endpoint or column we need from each one, and who on our side will lead each conversation. A short project-state README should travel with this memo so collaborators do not have to read the codebase before they can help.